why a request won't work, until I realize the request is going cross-domain or from HTTP to HTTPS, for example.

Whenever possible, I recommend avoiding cross-origin requests. You can end up with subtle differences in the way browsers handle them, which can cause more headaches. In particular, avoid HTTP to HTTPS cross-domain problems by running all your applications behind HTTPS. As discussed in chapter 28, that's a best practice anyway, and it'll help prevent a whole class of CORS headaches.

TIP Another (often preferable) option is to configure CORS policies in your reverse proxy or application gateway. You can configure Azure App Service with allowed origins, for example, so that you don't need to modify your application code.

Once I've established that I definitely need a CORS policy, I typically start with the `WithOrigins()` method. Then I expand or restrict the policy further, as need be, to provide cross-origin lockdown of my API while still allowing the required functionality. CORS can be tricky to work around, but remember, the restrictions are there for your safety.

Cross-origin requests are only one of many potential avenues attackers could use to compromise your app. Many of these are trivial to defend against, but you need to be aware of them and know how to mitigate them. In the next section we'll look at common threats and how to avoid them.

29.4 Exploring other attack vectors

So far in this chapter, I've described two potential ways attackers can compromise your apps—XSS and CSRF attacks—and how to prevent them. Both of these vulnerabilities regularly appear in the OWASP top ten list of